Pure Mathematics as a Language for Machine Learning and Data Analysis

N-20260302

Suppose a neural network turns a million parameters into one loss. Computing its full derivative sounds like a million separate experiments, yet one backward pass gives all million partial derivatives. Suppose four data points lie at the corners of a square. A loop in their neighborhood graph exists at one spatial scale and vanishes at another. Or suppose two probability distributions have the same variance but different centers: there is a way to transport one into the other that is visibly optimal, without solving a huge optimization problem.

These puzzles look unrelated until we ask what is being *carried across a map*. A perturbation can be pushed forward, a covector pulled backward, a topological cycle followed through an inclusion, a unit of probability mass coupled to its destination, and a symmetry carried through a predictor. Each operation has a precise compatibility law. The pleasure comes from discovering that the compatibility law often does the hard calculation for us.[1]

§1.1 Backpropagation as cotangent pullback

A gradient is usually displayed as a column of numbers. The more useful geometric object is the differential: a linear functional that evaluates the first-order effect of any perturbation. A forward neural network is a composite of maps between parameter spaces, activation spaces, and finally the one-dimensional loss space. If

$$L : Y \to \mathbb{R}$$

is the loss, then its differential at the output is a cotangent vector

$$dL_y \in T_y^* Y.$$

Backpropagation pulls this covector backward along the forward computational graph. In this sense the derivative operation used in training is contravariant:

$$f : X \to Y \quad \Longrightarrow \quad f_x^* : T_{f(x)}^* Y \to T_x^* X.$$

Forward maps push tangent perturbations forward; reverse-mode automatic differentiation pulls covectors backward.

For a concrete PyTorch-shaped layer, take

$$X \in \mathbb{R}^{b \times d}, \qquad W \in \mathbb{R}^{n \times d}, \qquad Y = XW^T \in \mathbb{R}^{b \times n}.$$

In indices,

$$Y_{\alpha i} = \sum_{k=1}^{d} X_{\alpha k} W_{ik}.$$

[1] The differential viewpoint follows MIT OCW's *Matrix Calculus for Machine Learning and Beyond*. For persistence see Edelsbrunner and Harer, *Computational Topology*; for transport see Peyré and Cuturi, *Computational Optimal Transport*.

The full Jacobian with respect to the weight matrix has entries

$$\frac{\partial Y_{\alpha i}}{\partial W_{jk}} = \delta_{ij} X_{\alpha k},$$

so its tensor shape is

$$(b, n, n, d).$$

This is the honest tensor object. Neural-network code does not materialize it because the upstream gradient

$$G = \frac{\partial L}{\partial Y} \in \mathbb{R}^{b \times n}$$

immediately contracts it:

$$\begin{aligned}\frac{\partial L}{\partial W_{jk}} &= \sum_{\alpha=1}^{b} \sum_{i=1}^{n} \frac{\partial L}{\partial Y_{\alpha i}} \frac{\partial Y_{\alpha i}}{\partial W_{jk}} \\ &= \sum_{\alpha=1}^{b} G_{\alpha j} X_{\alpha k}.\end{aligned}$$

Therefore

$$\boxed{\frac{\partial L}{\partial W} = G^T X.}$$

If one stores weights in the column convention $W_{\text{col}} \in \mathbb{R}^{d \times n}$ and writes $Y = X W_{\text{col}}$, the same contraction is

$$\frac{\partial L}{\partial W_{\text{col}}} = X^T G.$$

The categorical phrase "pull back the covector" is exactly this matrix multiplication: the high-order Jacobian collapses because it is being paired with a covector from the scalar loss.

Why reverse differentiation reverses arrows

Let $f : X \to Y$ and $g : Y \to Z$ be smooth maps. The differential moves a tiny displacement forward:

$$D(g \circ f)_x = Dg_{f(x)}\, Df_x.$$

A covector at $g(f(x))$ is a linear rule for evaluating such displacements. Applying it after the composite is the same as first pulling it through Dg and then through Df:

$$\boxed{(g \circ f)^*_x = f^*_x \circ g^*_{f(x)}.}$$

The order reverses automatically. This identity is the precise meaning of reverse-mode differentiation; it is a property of dual maps.

For a small calculation, define

$$u_1 = x_1 + x_2, \qquad u_2 = x_1 x_2, \qquad L = u_1^2 + \tfrac{1}{2} u_2^2.$$

At $(x_1, x_2) = (1, 2)$, the forward pass gives $(u_1, u_2) = (3, 2)$ and $L = 11$. The loss first supplies the covector $(2u_1, u_2) = (6, 2)$. Pull it through

$$Du = \begin{pmatrix} 1 & 1 \\ x_2 & x_1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 2 & 1 \end{pmatrix}$$

to obtain

$$\boxed{\nabla_x L = (Du)^T \begin{pmatrix} 6 \\ 2 \end{pmatrix} = \begin{pmatrix} 10 \\ 8 \end{pmatrix}.}$$

We could expand $L = (x_1 + x_2)^2 + \frac{1}{2}x_1^2 x_2^2$ and differentiate directly; it gives the same answer. The two computations differ in what they remember: the backward calculation reuses the intermediate values of the forward computation.

There is also a useful pairing identity behind every implementation. If a forward perturbation is δx, then

$$\langle dL_y, Df_x(\delta x)\rangle = \langle (Df_x)^* dL_y, \delta x\rangle.$$

Both sides are the same first-order change in the loss. When Df is too large to write as a matrix, this identity defines the operation that the backward pass must compute. It also makes a finite-difference sanity check possible: replacing x by $x + h\delta x$ should change L by $h\,\langle (Df_x)^* dL, \delta x\rangle + O(h^2)$.

The Adjoint-State Method

Reverse-mode differentiation is even more striking when the layer is the solution of an equation. Suppose a parameter vector θ determines an invertible matrix and a right-hand side, and define $u(\theta)$ implicitly by

$$A(\theta)u(\theta) = b(\theta).$$

We care about one scalar objective $J(\theta) = \ell(u(\theta), \theta)$. Differentiating the linear system in a direction $\delta\theta$ yields

$$A\,\delta u = \delta b - (\delta A)u.$$

It looks as though we must solve another system for δu for *every* parameter. But the differential of the objective is only

$$\delta J = \nabla_u \ell^T \delta u + D_\theta \ell[\delta\theta].$$

Solve instead the single *adjoint system*

$$\boxed{A^T \lambda = \nabla_u \ell.}$$

Then the unwanted δu disappears:

$$\begin{aligned} \delta J &= \lambda^T A\,\delta u + D_\theta \ell[\delta\theta] \\ &= \boxed{\lambda^T(\delta b - (\delta A)u) + D_\theta \ell[\delta\theta].} \end{aligned}$$

One forward solve for u and one backward solve for λ give directional derivatives with respect to arbitrarily many parameters. This is the adjoint method used in differentiable physics and PDE-constrained optimization; backpropagation through a feedforward network is its finite computational-graph counterpart.

Here is a two-by-two check:

$$A(t) = \begin{pmatrix} 1 & t \\ 0 & 1 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad J(t) = \frac{1}{2}\|u(t)\|^2.$$

Solving directly gives $u(t) = (1 - t, 1)^T$, so

$$J'(t) = -(1 - t).$$

For the adjoint method, $A(t)^T\lambda = u(t)$ implies $\lambda_1 = 1 - t$. Since

$$A'(t)u(t) = \begin{pmatrix} 1 \\ 0 \end{pmatrix},$$

the boxed formula gives $J'(t) = -\lambda_1 = -(1-t)$ again. The little example demonstrates the point of a large theory: it is the *adjoint variable*, not a gigantic derivative array, that answers a scalar sensitivity question.

§1.2 Geometric data analysis

The same question reappears in data analysis: which mathematical language becomes necessary when coordinates alone no longer describe the structure?

A point cloud is often treated as a finite sample from a hidden space. Topological data analysis makes this idea precise by replacing the cloud with a filtered simplicial complex. For a scale parameter ε, one forms for instance the Vietoris–Rips complex

$$VR_\varepsilon(X) = \{\sigma \subseteq X : \ d(x,y) \le \varepsilon \text{ for all } x, y \in \sigma\}.$$

As ε grows, connected components merge, loops appear and disappear, and higher-dimensional holes may be born and die. Persistent homology records this evolution. The chain complex viewpoint is governed by the boundary identity

$$\partial_{k-1} \circ \partial_k = 0,$$

and homology is

$$H_k = \ker \partial_k / \operatorname{im} \partial_{k+1}.$$

This quotient is the mathematical form of the phrase "holes that are cycles but not boundaries."

Example 1.2.1 (A square-shaped hole with a finite lifetime)

Take the four vertices of the unit square with Euclidean distance. When $1 \le \varepsilon < \sqrt{2}$, their Vietoris–Rips complex is a four-edge cycle without any filled triangles. Its first homology over a field has dimension one. When $\varepsilon \ge \sqrt{2}$, both diagonals appear, so the complex is the complete 3-simplex and H_1 vanishes. The one-dimensional homology class therefore persists over $[1, \sqrt{2})$ under this filtration convention.

Four points, one hole, and one diagonal

Why can a loop be visible at one resolution and disappear at another? Work with coefficients in $\mathbb{F}_2$, so that reversing the orientation of an edge makes no difference. Label the corners of the unit square $0, 1, 2, 3$ in cyclic order. At scale $1 \le \varepsilon < \sqrt{2}$ the Rips complex contains the four edges but neither diagonal. Consider the 1-chain

$$c = [01] + [12] + [23] + [30].$$

The boundary of an edge is the sum of its two endpoints, hence

$$\partial_1 c = (0+1) + (1+2) + (2+3) + (3+0) = 0 \quad \text{in } \mathbb{F}_2.$$

It is a cycle. But the complex has *no triangles*, so $\operatorname{im}\partial_2 = 0$: there is no two-dimensional chain with boundary c. Consequently

$$\boxed{H_1(VR_\varepsilon(X);\mathbb{F}_2) \cong \mathbb{F}_2, \qquad 1 \le \varepsilon < \sqrt{2}.}$$

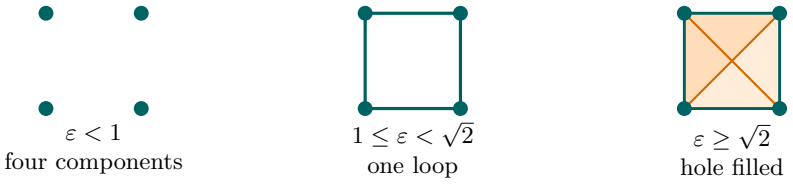


Figure 1.1: The Vietoris–Rips filtration of four square corners. Edges appear at distance 1; diagonals and all triangles appear at distance $\sqrt{2}$. The loop therefore survives for precisely the interval $[1, \sqrt{2})$. The rightmost complex also includes the tetrahedral 3-simplex, whose two-dimensional faces are represented by the filled regions.

When the diagonal [02] appears, the two triangles [012] and [023] are present. Their boundaries add to

$$\partial_2([012] + [023]) = [01] + [12] + [20] + [02] + [23] + [30] = c,$$

because the diagonal occurs twice and cancels in $\mathbb{F}_2$. The same chain that was a nontrivial hole has become a boundary. Nothing about the four points changed; only the scale of the question did. A persistent barcode draws the resulting H_1 class as one horizontal interval with birth 1 and death $\sqrt{2}$.

Why this picture survives small measurement errors

Suppose the point x_i moves to y_i with $d(x_i, y_i) \le \delta$. For every pair,

$$\bigl|d(x_i, x_j) - d(y_i, y_j)\bigr| \le 2\delta$$

by the triangle inequality. Thus every edge, triangle, or higher simplex present at scale ε in the first Rips complex is present at scale $\varepsilon + 2\delta$ in the second:

$$VR_\varepsilon(X) \longrightarrow VR_{\varepsilon+2\delta}(Y).$$

The reverse comparison has the same form. Such inclusions give a 2δ-interleaving of the two filtrations, so their persistence diagrams can be matched with bottleneck error at most 2δ (under the standard finite-filtration conventions). In particular, a loop with a lifetime much longer than 4δ cannot be matched to the diagonal by a perturbation of this size: its lifetime exceeds twice the bottleneck error.

This is the practical reason to measure *persistence* rather than count holes at one chosen resolution. One scale may be capricious; a long interval survives a quantified amount of noise. The general stability theorem of Cohen-Steiner, Edelsbrunner and Harer develops this idea for sublevel-set persistence. For finite point clouds, the simple edge-distance argument above already explains the stability mechanism.

§1.3 Manifold hypothesis and embeddings

The manifold hypothesis says that high-dimensional data often concentrate near a lower-dimensional manifold $M \subseteq \mathbb{R}^N$. Whitney's embedding theorem guarantees a smooth embedding of a smooth d-manifold into $\mathbb{R}^{2d}$. This is an existence theorem for smooth coordinates; it does not imply that a finite noisy dataset lies on such a manifold, or that a learned representation will recover an embedding. These conclusions require separate assumptions on sampling, geometry, and the optimization objective.

The practical warning is that an embedding preserves the manifold as a smooth object, but not necessarily distances, probability densities, or task-relevant semantics. PCA preserves a linear subspace model; manifold learning tries to preserve local neighborhoods; representation learning tries to learn coordinates useful for a loss function.

A Circle That Defeats PCA

Take points uniformly distributed on the unit circle,

$$X = (\cos\Theta, \sin\Theta), \qquad \Theta \sim \mathrm{Unif}[0, 2\pi).$$

The support is a smooth one-dimensional manifold. Yet its mean is zero and its covariance is

$$\mathrm{Cov}(X) = \begin{pmatrix} \mathbb{E}\cos^2\Theta & \mathbb{E}\sin\Theta\cos\Theta \\ \mathbb{E}\sin\Theta\cos\Theta & \mathbb{E}\sin^2\Theta \end{pmatrix} = \tfrac{1}{2} I_2.$$

Every line through the origin explains exactly the same variance. Therefore PCA has *no preferred first principal direction*, even though the data are intrinsically one-dimensional. The dimension of a linear span and the dimension of a curved manifold answer different questions.

There is a second difficulty. If we try to use the first coordinate $x = \cos\theta$ as a global parameter, it assigns the same value to θ and $-\theta$. At the leftmost and rightmost points, projection onto the horizontal axis has vanishing tangential derivative and fails to be a local coordinate. A single linear coordinate cannot traverse a circle once without identifying points.

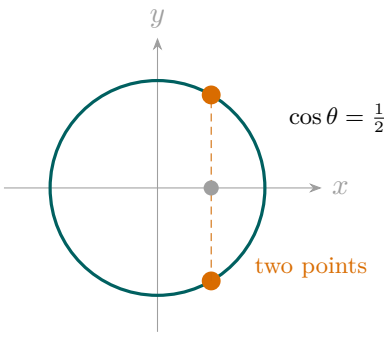


Figure 1.2: The circle has intrinsic dimension one but covariance $\frac{1}{2} I_2$, so its PCA directions are tied. The two orange points are distinct, yet their projection onto the x-axis is identical.

A genuinely local coordinate can be obtained by the half-angle substitution

$$t = \tan\frac{\theta}{2}, \qquad (\cos\theta, \sin\theta) = \left(\frac{1 - t^2}{1 + t^2}, \frac{2t}{1 + t^2}\right).$$

As t runs through $\mathbb{R}$, this parametrizes the circle except the point $(-1, 0)$, which is reached only as $|t| \to \infty$. A second chart handles that missing point. This illustrates exactly why differential geometry uses *atlases* of charts, and why a globally continuous one-to-one latent coordinate can be topologically impossible for a circular data manifold.

§1.4 Optimal transport and stochastic dynamics

For probability measures μ, ν on a metric space, the p-Wasserstein distance is

$$W_p(\mu,\nu) = \left(\inf_{\pi \in \Pi(\mu,\nu)} \int d(x,y)^p \, d\pi(x,y) \right)^{1/p}.$$

Here $\Pi(\mu,\nu)$ is the set of couplings with marginals μ and ν. The dual viewpoint is equally important. For $p = 1$,

$$W_1(\mu,\nu) = \sup_{\|f\|_{\mathrm{Lip}} \le 1} \left(\int f \, d\mu - \int f \, d\nu \right).$$

The uncrossing trick

Let two equally weighted particles sit at -1 and 1, with destinations 0 and 2:

$$\mu = \tfrac{1}{2}\delta_{-1} + \tfrac{1}{2}\delta_1, \qquad \nu = \tfrac{1}{2}\delta_0 + \tfrac{1}{2}\delta_2.$$

Sending -1 to 0 and 1 to 2 costs $\frac{1}{2}(1^2 + 1^2) = 1$; crossing the assignments costs $\frac{1}{2}(3^2 + 1^2) = 5$. The reason extends beyond this example. If $x_1 < x_2$ and $y_1 < y_2$,

$$(x_1 - y_1)^2 + (x_2 - y_2)^2 - (x_1 - y_2)^2 - (x_2 - y_1)^2 = -2(x_2 - x_1)(y_2 - y_1) \le 0.$$

Thus uncrossing any two matched pairs cannot raise quadratic transport cost.

Write $F^{-1}(u) = \inf\{x : F(x) \ge u\}$ for a distribution's quantile function. Pairing the same uniform random number $U \in (0,1)$ across two distributions gives $X = F_\mu^{-1}(U)$ and $Y = F_\nu^{-1}(U)$. The no-crossing argument first proves optimality for finite ordered point clouds; approximating general one-dimensional distributions gives

$$\boxed{W_p^p(\mu,\nu) = \int_0^1 |F_\mu^{-1}(u) - F_\nu^{-1}(u)|^p \, du, \quad p \ge 1,}$$

when the p-th moments are finite. A minimization over every possible joint distribution has collapsed to one integral. Empirically, this says to sort both samples and pair their order statistics.

For $p = 2$ there is a useful one-dimensional closed form. Let

$$\mu = \mathcal{N}(a, \sigma_1^2), \qquad \nu = \mathcal{N}(b, \sigma_2^2), \qquad \sigma_1, \sigma_2 > 0.$$

In one dimension the optimal map between these two Gaussians is affine and monotone, so write

$$T(x) = \alpha x + \beta.$$

The condition $T_{\#}\mu = \nu$ forces

$$\alpha = \frac{\sigma_2}{\sigma_1}, \qquad \beta = b - \alpha a.$$

If $X = a + \sigma_1 Z$ with $Z \sim \mathcal{N}(0,1)$, then

$$T(X) = b + \sigma_2 Z.$$

Thus

$$\begin{aligned} W_2^2(\mu,\nu) &= \mathbb{E}[(X - T(X))^2] \\ &= \mathbb{E}[((a-b) + (\sigma_1 - \sigma_2)Z)^2] \\ &= (a-b)^2 + (\sigma_1 - \sigma_2)^2. \end{aligned}$$

This formula is small, but it explains a lot of geometric language around generative models: matching Gaussians means matching both their centers and their spreads.

A curved-looking space of measures hides a flat plane

The Gaussian formula does something more surprising than calculate a distance. Fix two one-dimensional Gaussian laws with means a, b and positive standard deviations σ_0, σ_1. Their optimal coupling uses the *same* standard Gaussian Z:

$$X_0 = a + \sigma_0 Z, \qquad X_1 = b + \sigma_1 Z.$$

Move each paired particle at constant speed:

$$X_s = (1-s)X_0 + sX_1 = \big((1-s)a + sb\big) + \big((1-s)\sigma_0 + s\sigma_1\big)Z.$$

The law μ_s is Gaussian for every $0 \le s \le 1$. Writing $a_s = (1-s)a + sb$ and $\sigma_s = (1-s)\sigma_0 + s\sigma_1$, the one-dimensional transport formula gives

$$\begin{aligned} W_2^2(\mu_s,\mu_t) &= (a_s - a_t)^2 + (\sigma_s - \sigma_t)^2 \\ &= |s-t|^2((a-b)^2 + (\sigma_0 - \sigma_1)^2). \end{aligned}$$

Thus

$$\boxed{W_2(\mu_s,\mu_t) = |s-t|\, W_2(\mu_0,\mu_1).}$$

We have constructed a constant-speed Wasserstein geodesic without invoking geodesic equations. In fact, the space of one-dimensional Gaussian distributions is isometric, under W_2, to the Euclidean half-plane with coordinates (mean, standard deviation) and $\sigma > 0$.

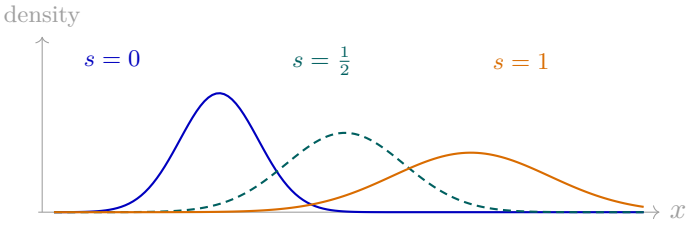


Figure 1.3: Three points of a Wasserstein geodesic between $\mathcal{N}(-1.6, 0.5^2)$ and $\mathcal{N}(1.6, 1^2)$. The intermediate law is $\mathcal{N}(0, 0.75^2)$: both the mean and the standard deviation interpolate linearly. This is the geometry of optimal coupling, distinct from merely adding Gaussian noise.

Diffusion models can be read through stochastic differential equations

$$dX_t = b(t, X_t)\, dt + \sigma(t)\, dW_t.$$

The density p_t evolves according to a Fokker–Planck equation, and the reverse-time dynamics involve the score $\nabla_x \log p_t(x)$. Thus score-based generative modeling is an attempt to learn the vector field that reverses a noisy transport of probability mass.

For DDPMs the training objective is usually written as a variational lower bound, or in its simplified form as a denoising mean-square loss. It is not literally a Wasserstein objective. Still, each reverse step compares Gaussian conditional distributions, and for Gaussians the geometry above says that a squared displacement of means and a mismatch of variances are exactly the ingredients of W_2^2. The ELBO uses KL divergences between Gaussian conditionals; when the variance is fixed or separately prescribed, minimizing that KL has the same practical center-matching behavior as minimizing the corresponding Gaussian W_2 term. The distinction matters: an optimal-transport cost minimizes over couplings, whereas the DDPM training objective comes from a likelihood-based variational construction. Similar quadratic penalties for fixed-variance Gaussian comparisons do not make the two optimization problems equivalent.

Score as a Denoising Direction

Let X_0 be any random vector in $\mathbb{R}^d$ with finite first moment. Corrupt it with Gaussian noise:

$$Y = X_0 + \sqrt{t}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \quad t > 0.$$

Its density is the Gaussian convolution

$$p_t(y) = \int (2\pi t)^{-d/2} e^{-\|y-x\|^2/(2t)}\, p_0(dx).$$

Differentiate *this* expression, using $\nabla_y e^{-\|y-x\|^2/(2t)} = -(y-x)e^{-\|y-x\|^2/(2t)}/t$:

$$t\nabla_y p_t(y) = \int (x-y)\,\varphi_t(y-x)\,p_0(dx),$$

$$t\nabla_y \log p_t(y) = \frac{\int x\varphi_t(y-x)\,p_0(dx)}{\int \varphi_t(y-x)\,p_0(dx)} - y.$$

The quotient is just the conditional expectation of the clean signal given its noisy observation. Therefore

$$\boxed{\mathbb{E}[X_0 \mid Y = y] = y + t\,\nabla_y \log p_t(y).}$$

This is Tweedie's formula. It explains the score as a geometrically meaningful vector field: t times the score is the correction from a noisy point to its *posterior mean* clean point. It is a theorem about conditional expectation, independent of any neural network.

Here is a complete discrete example. Let X_0 be equally likely to be $-a$ or $+a$. After observing $Y = y$, the posterior odds are

$$\frac{\Pr(X_0 = a \mid y)}{\Pr(X_0 = -a \mid y)} = \exp\left(\frac{2ay}{t}\right).$$

Hence

$$\mathbb{E}[X_0 \mid y] = a\tanh\frac{ay}{t}, \qquad \boxed{\frac{d}{dy}\log p_t(y) = \frac{a\tanh(ay/t) - y}{t}.}$$

For small noise, positive observations are corrected toward $+a$ and negative ones toward $-a$; near $y = 0$, the two explanations compete. As noise increases, the two peaks merge.

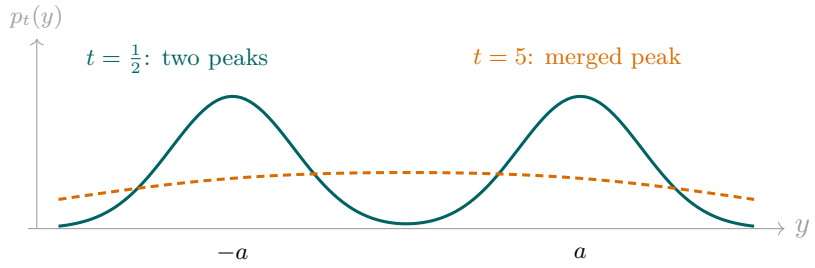


Figure 1.4: Two point masses at $\pm a$ with $a = 2$, blurred by Gaussian noise. The colored curves are the exact Gaussian-mixture densities at $t = \frac{1}{2}$ and $t = 5$, plotted with the same vertical multiplier for legibility. The score field is the logarithmic derivative of these changing distributions.

Optimal transport and diffusion can both describe movement of probability mass, but their mechanisms differ: the former chooses a coupling to minimize displacement, while heat diffusion continually mixes in new noise. The denoising formula makes precise how the latter noisy evolution can still carry information about the original distribution.

§1.5 Symmetry, equivariance, and category-like thinking

If a group G acts on an input space and an output space, a map F is equivariant if

$$F(gx) = gF(x).$$

Convolutional neural networks exploit translation equivariance; graph neural networks exploit permutation equivariance; gauge-equivariant networks try to respect local choices of frame on a bundle.

Here is the matrix calculation behind the slogan. Let the graph be the cycle C_4, with adjacency matrix

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix},$$

and let P be the permutation matrix rotating the four vertices by $90°$. Since P is a graph automorphism,

$$PA = AP.$$

The same is true for the usual normalized adjacency $\widehat{A}$, because normalization is built from A and the degree matrix, both invariant under the same rotation:

$$P\widehat{A} = \widehat{A}P.$$

A standard graph neural-network layer is

$$H^{(l+1)} = \sigma(\widehat{A}H^{(l)}W),$$

where σ acts entrywise. If the input features are rotated, then

$$\begin{aligned} F(PH^{(l)}) &= \sigma(\widehat{A}PH^{(l)}W) \\ &= \sigma(P\widehat{A}H^{(l)}W) \\ &= P\sigma(\widehat{A}H^{(l)}W) \\ &= PH^{(l+1)}. \end{aligned}$$

Equivariance is therefore not only a philosophical commitment to symmetry. In this example it is the algebraic fact that the permutation matrix commutes with the graph operator.

Functoriality has a testable meaning. A continuous map $f : X \to Y$ induces a homomorphism $H_k(f) : H_k(X) \to H_k(Y)$, and $H_k(g \circ f) = H_k(g) \circ H_k(f)$. This is a compatibility law between topological maps and homology, not an explanation of intelligence. Equivariance of a graph layer is likewise an explicit compatibility statement, expressed here by the commuting identity $F(PH) = PF(H)$.

An equivariant map can be made by averaging

There is a pleasing construction when the symmetry group is finite. Suppose G acts linearly on input and output spaces by representations ρ_{in} and ρ_{out}. Take *any* function h, possibly a function with no symmetry at all, and define

$$\boxed{(\mathcal{R}h)(x) = \frac{1}{|G|} \sum_{g \in G} \rho_{\text{out}}(g^{-1}) h\big(\rho_{\text{in}}(g)x\big).}$$

Why should this be equivariant? Apply the definition to a transformed input $\rho_{\text{in}}(k)x$, and rename the summation index $u = gk$:

$$\begin{aligned}(\mathcal{R}h)(\rho_{\text{in}}(k)x) &= \frac{1}{|G|} \sum_{g} \rho_{\text{out}}(g^{-1}) h(\rho_{\text{in}}(gk)x) \\ &= \rho_{\text{out}}(k) \frac{1}{|G|} \sum_{u} \rho_{\text{out}}(u^{-1}) h(\rho_{\text{in}}(u)x) \\ &= \rho_{\text{out}}(k)(\mathcal{R}h)(x).\end{aligned}$$

The proof consists of one change of summation variables. The group operation guarantees that the average sees every transformed input exactly once.

For example, let $G = \{e, s\}$ swap the coordinates of $\mathbb{R}^2$ and take

$$h(x_1, x_2) = (x_1^2 + x_2,\, 0).$$

The raw function does not commute with the swap. Its symmetrization is

$$(\mathcal{R}h)(x_1, x_2) = \frac{1}{2}\big(x_1^2 + x_2,\; x_2^2 + x_1\big),$$

which clearly swaps its two outputs whenever the inputs are swapped. This construction is often called *group averaging* or a Reynolds-type operator. It is an explicit way to turn a desired symmetry law into an exact property of a model.

There is a distinction worth keeping in the graph example. A *graph automorphism* satisfies $PAP^T = A$, so it may act on the node features while the adjacency matrix stays fixed. An arbitrary relabeling P usually changes the displayed adjacency matrix too. Under a simultaneous relabeling of graph and features, however,

$$\widehat{PAP^T} = P\widehat{A}P^T, \qquad \boxed{F(PAP^T, PH) = PF(A, H).}$$

This is permutation equivariance as a statement about the *representation of the same graph*, even when the permutation is not an automorphism of that graph.

A common law for maps of different kinds

The contravariance of backpropagation and the covariance of homology are different laws. A continuous map of spaces $f : X \to Y$ induces

$$H_k(f) : H_k(X) \to H_k(Y), \qquad H_k(g \circ f) = H_k(g) \circ H_k(f).$$

The directions here are *forward*: a chain is sent to the image chain. For cotangent vectors, dualizing reverses composition. The distinction is concrete. If an inclusion of simplicial complexes fills a hole, a nonzero cycle class may be carried *forward* to zero, whereas a sensitivity to an output is carried *backward* to an input.

The common principle is functoriality: what matters about a transformation is what it does to the structure under study, and a composite transformation must act compatibly with its two pieces. In machine learning this is useful only when the acting map and the preserved structure are named explicitly; otherwise the language has no computational content.

§1.6 Maps that remember the right information

The group-averaging construction hides one last satisfying fact. If h is already equivariant, every summand in $\mathcal{R}h$ equals $h(x)$, and therefore $\mathcal{R}h = h$. Because the first application of $\mathcal{R}$ makes *any* function equivariant, applying it again changes nothing:

$$\boxed{\mathcal{R}^2 = \mathcal{R}.}$$

Thus averaging is a *projection* in the space of functions, even when we have not chosen an inner product there. It takes an arbitrary predictor and projects it onto a class of predictors obeying an exact symmetry law. With a suitable invariant inner product, it becomes the familiar orthogonal projection onto the fixed subspace. A construction from group theory has become a geometric operation on models.

The other calculations have a similarly concrete payoff. The differential does not ask us to store a huge Jacobian: the cotangent pullback tells us which contraction to perform. A loop around four points becomes a boundary at a precise scale, and stability turns that observation into a noise-tolerant feature. Pairing quantiles solves the one-dimensional transport problem by forbidding crossings. Differentiating a Gaussian convolution turns an intractable-looking posterior expectation into a derivative of an observable density.

There is no need to treat these methods as interchangeable. Each has a different domain and a different invariant. What makes them powerful is that the difficult calculation is replaced by a structural identity whose assumptions can be checked and whose consequences can actually be computed.